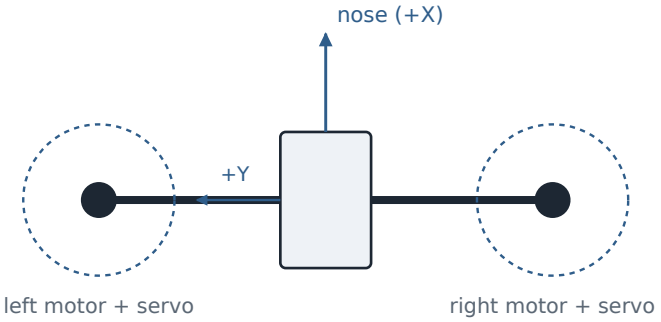
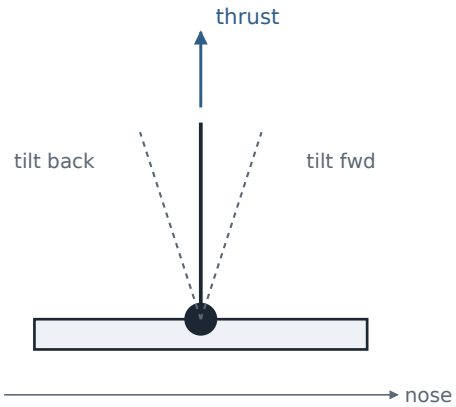
BiCopter Flight Controller

Firmware for a tilt-rotor bicopter on the ESP32-S3 PowerFeather



Top view



Side view, one motor

This guide explains what the project is, how the controller works and how to rebuild it step by step: from wiring and the first bench checks to tuning the PID gains on the real model. It finishes with the problems that came up during testing and the sources the design draws on.

Mohammed Abdelsabour
October 2026

1. What this project is

A bicopter is a multicopter with only two rotors. Quadcopters can steer purely by changing motor speeds, but two rotors are not enough for that, so each motor is mounted on a servo that tilts it forward and back. The result is a compact, efficient airframe that is also harder to control: one axis is driven by fast motors, the other two by much slower servos.

This project is the flight controller for such a model. It runs on an ESP32-S3 PowerFeather, reads a standard hobby radio receiver and an MPU6050 motion sensor, and drives two ESCs and two servos. The firmware was written from scratch for this airframe, with three priorities in this order: that it is safe to test with, that it can be understood by reading it, and that it flies well.

What it does

- Self-levelling flight: roll and pitch sticks set an angle, and the model returns to level when they are released. With the yaw stick centred it also holds its heading.
- A 500 Hz control loop with a cascaded angle and rate controller on each axis.
- Arming, disarming and level calibration from the sticks, because a 4-channel radio has no switches.
- Failsafe, crash detection, automatic disarm and a hardware watchdog.
- A USB console with live data streams, servo trims and PID tuning without re-flashing.

What it does not do

It holds attitude and heading, not position. There is no GPS, optical flow or altitude sensor, so the pilot controls height with the throttle and corrects slow drift by hand. Section 9 suggests how that could be added.

Hardware used

Part	Model	Role
Flight controller	ESP32-S3 PowerFeather	dual core 240 MHz, native USB, battery management
Motion sensor	MPU6050 on a GY-521 board	3-axis gyro and 3-axis accelerometer over I2C
Radio	FlySky FS-i6 with FS-iA6B	4 channels, one PWM signal per channel
Tilt servos	2 × MG996R	tilt each motor forward and back
Propulsion	2 × brushless motor + ESC	standard PWM ESCs at 400 Hz
Servo supply	5–6 V UBEC	keeps servo current away from the controller

2. How a bicopter flies

Each of the four actuators has a clear job, but most of them do two jobs at once. The firmware's mixer combines the controller's three corrections and the throttle into the four outputs:

```
left motor  = throttle + roll          left servo  = pitch - yaw
right motor = throttle - roll          right servo = pitch + yaw
```

- **Roll** (tipping left or right) comes from running one motor faster than the other. This is the fastest axis because motors respond within a few milliseconds.
- **Pitch** (nose up or down) comes from tilting both motors the same way. Tilting the thrust forward pushes the top of the model forward, which lowers the nose.
- **Yaw** (turning) comes from tilting the motors in opposite directions, so one pushes forward and the other back.

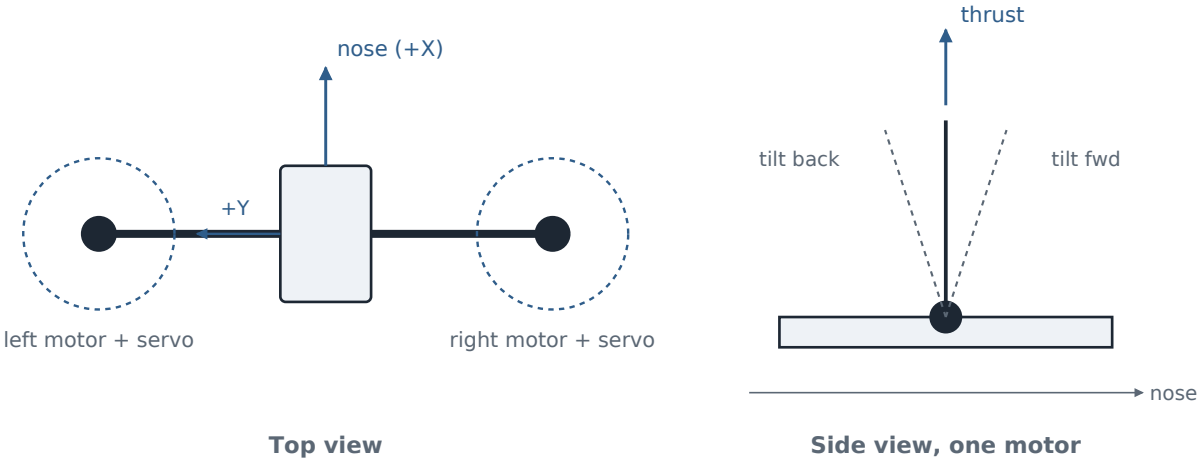


Figure 1. The airframe. Roll is controlled by motor speed, pitch and yaw by servo tilt.

Figure 2 shows the real model. At each end of the arm an MG996R servo carries a 3D-printed mount, and the motor sits on top of that mount, so the servo tilts the whole motor and propeller about the arm. The flight controller, the IMU and the battery are on the centre of the arm. The printed parts are existing Thingiverse designs, credited in hardware/README.md.

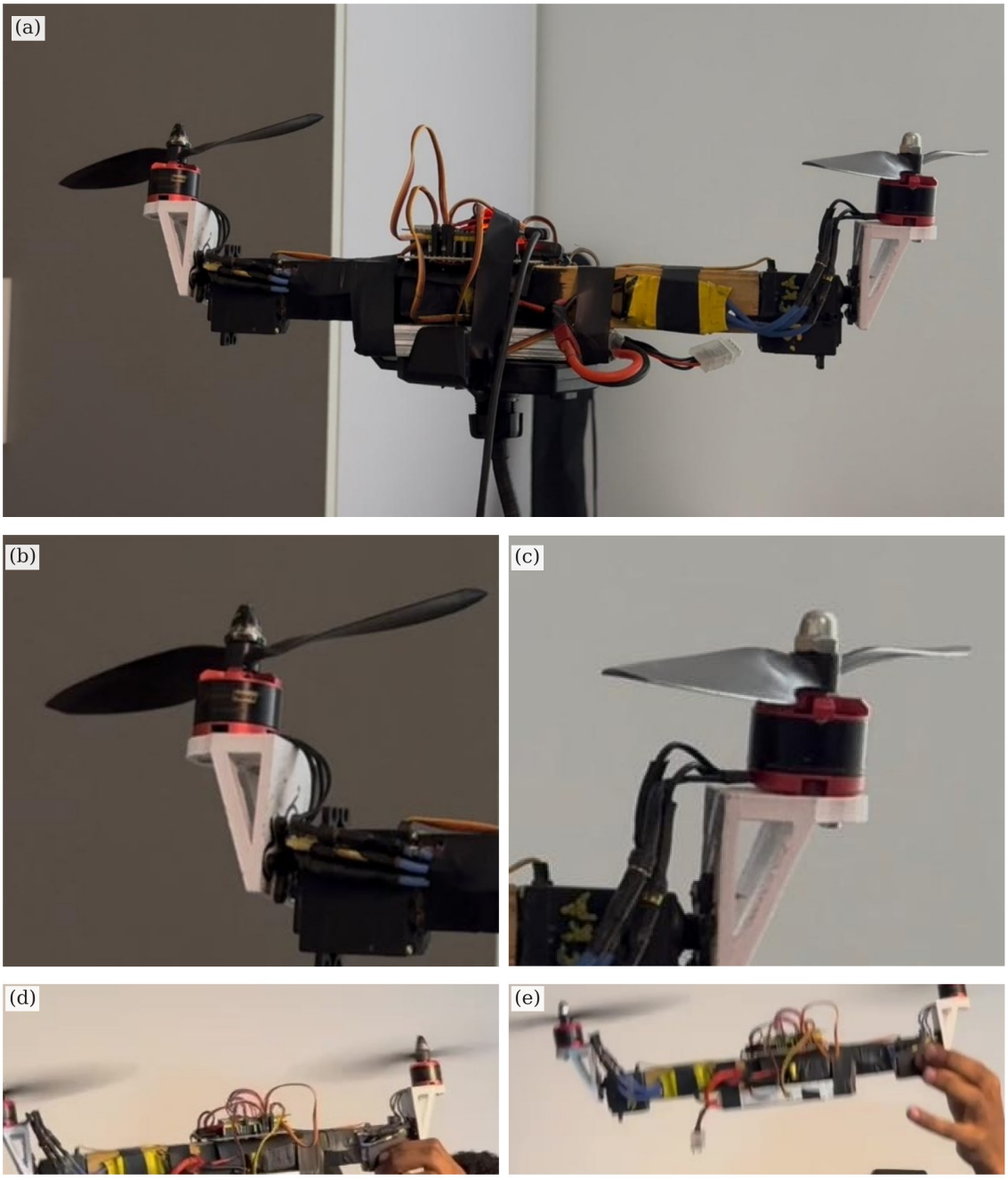


Figure 2. The bicopter built in this project: (a) the complete model on its test stand, (b) the left and (c) the right tilt unit, each a motor on a printed mount turned by an MG996R servo, (d, e) the model held by hand with both motors running, during a balancing test.

Axes and signs

The whole firmware uses one body frame: X points to the nose, Y to the left motor and Z up. Positive roll means the right side goes down, positive pitch means the nose goes down, and positive yaw means the nose turns left. Getting a single one of these signs wrong is the most common reason a new build flips the moment it leaves the ground, which is why the setup procedure checks each one separately.

Why it is harder than a quadcopter

The MG996R servos take about 0.17 s to move 60°, and their gears have some free play. Pitch and yaw therefore react far more slowly than roll, and the gains on those axes have

to stay modest. The two axes are also coupled: when the roll controller runs one motor faster, that motor's reaction torque tries to turn the model, and the yaw controller has to tilt the servos to cancel it.

3. Firmware design

The firmware is a PlatformIO project for the Arduino framework. The sketch itself only connects the pieces and runs the loop. Each job lives in its own small library under `lib/`, with one header and one source file. None of the libraries read the project configuration directly: each takes a small settings struct, which the sketch fills in from `Config.hpp`. That makes every part reusable and testable on its own.

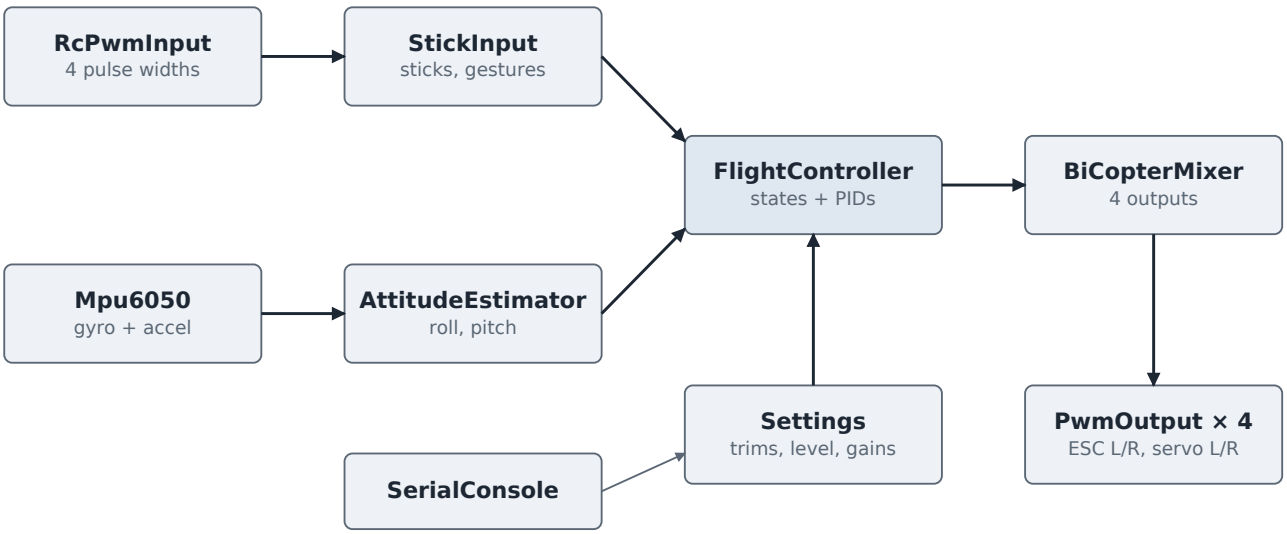


Figure 3. Data flow in one pass of the control loop.

Library	What it does
Mpu6050	Reads gyro, accelerometer and temperature in one 14-byte burst. Calibrates the gyro at start-up and remaps the sensor axes to the body frame.
RcPwmInput	Measures the four receiver pulses with the ESP32-S3's MCPWM capture hardware, so the timing does not depend on interrupt delays. A median of three pulses removes glitches.
StickInput	Converts pulses to stick values, learns where each stick rests, detects the receiver failsafe and recognises the arm, disarm and level gestures.
AttitudeEstimator	Estimates roll and pitch with a Mahony filter: the gyro for fast changes, the accelerometer to remove slow drift, and an integral term that learns the gyro's zero error in flight.
Pid / Pt1Filter / BiquadFilter	The rate controller and the low pass filters used on the gyro, the accelerometer, the D term and the servo commands.
LeadCompensator	Optional phase lead for the servo commands, so a slow servo starts moving sooner. Off by default.
BiCopterMixer / PwmOutput	Turn the corrections into four pulse widths and generate them on the LEDC peripheral: 400 Hz for the ESCs, 100 Hz for the servos.
FlightController	The state machine (disarmed, armed, failsafe, fault), the safety checks and the control law.
Settings / SerialConsole	Keep trims, level and tuned gains in flash, and provide the USB console.

The control loop

The loop runs every 2 ms. Each pass reads the sensor, updates the attitude estimate, reads the sticks, runs the state machine and the controllers, and writes the outputs. Reading the sensor takes about 0.4 ms of the 2 ms, and the console reports the longest pass so that a slow change shows up straight away.

Each axis uses two loops in series. The outer loop compares the angle the pilot wants with the measured angle and asks for a rotation rate. The inner loop compares that rate with the gyro and drives the actuators. The inner loop does most of the work against disturbances; the outer loop only has to decide where level is.

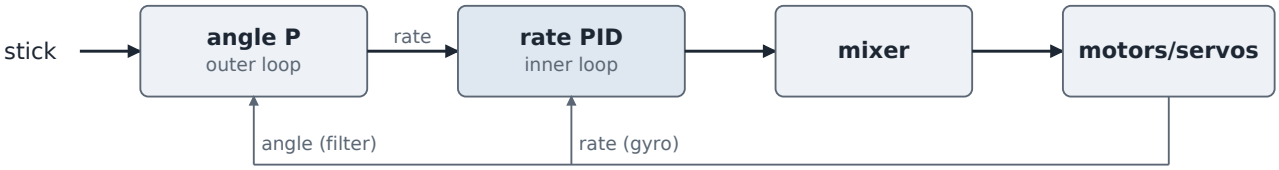


Figure 4. Cascaded control on one axis.

Attitude estimate

The gyro measures rotation rate very accurately but slowly drifts. The accelerometer knows where down is, but every vibration and manoeuvre disturbs it, and on a bicopter it mostly sees the direction of the motor thrust, which moves with every servo correction. It can only be trusted slowly.

The firmware uses a Mahony filter on a quaternion. The gyro is integrated as the main source. The accelerometer, after a 10 Hz second order low pass, steers the estimate slowly towards "down" (a 3 s time constant). An integral term learns the gyro's zero error in flight, so the slow correction does not leave a steady angle error behind:

```
error   = measured_down x estimated_down              (cross product)
bias   += ki * trust * error * dt                       (learns the gyro zero error)
rate    = gyro + kp * trust * error + bias              kp = 1 / 3 s, ki = kp / 15 s
q      += 0.5 * q * (0, rate) * dt                      (then normalise)
```

How much the accelerometer is trusted fades smoothly from full at exactly 1 g to nothing at 0.8 g or 1.2 g, because outside that range it is measuring the model's motion as well as gravity. In a simulated hover with a 0.5 deg/s gyro error and prop vibration, this cut the angle error from about 1.5 deg to 0.1 deg compared with the simple complementary filter used before.

Filters

The sensor's own low pass filter is set to 44 Hz. On top of that, the gyro signal is filtered at 80 Hz for roll, 40 Hz for pitch and 25 Hz for yaw, and the servo commands at 30 Hz. The servos are updated at 100 Hz. Every filter adds a little delay, so the servo axes use the lightest filtering that still keeps vibration from making the servos buzz inside their gear play.

Take-off detection

Sitting on the ground with the motors spooling, the frame cannot respond to corrections. An I term running there keeps adding correction against the ground, and in testing it tilted the servos 20-25 deg before lift-off, so the model shot forward as it left the ground.

The I terms, heading hold and roll on the motors therefore only switch on once the throttle reaches the lift-off throttle, and switch off again only near idle.

4. Safety design

A bicopter on the bench can hurt someone, so the firmware starts from "motors off" and only leaves that state when every condition is right.

Situation	What the firmware does
Power-up	ESC outputs go to 1000 µs (motor off) before anything else runs.
Arming	Refused unless the radio is valid, the throttle is low, the model is within 25° of level, the gyro is calibrated and the stick centres have been learned. The console prints the reason.
Radio lost	Motors stop. To fly again the radio must come back with the throttle low, and the pilot has to arm again.
Crash	Tilted past 70° for half a second: disarm.
Forgotten model	Armed at idle for 15 s: disarm.
Sensor failure	25 failed reads in a row (50 ms): motors stop until reboot.
Software hang	A 300 ms watchdog resets the chip. All outputs go low, so the ESCs stop.
On the ground	At idle, roll corrections are not sent to the motors, so the model cannot tip itself over before take-off. The I terms are held at zero until the throttle is up.

Receiver failsafe. When the FS-iA6B loses the transmitter it keeps repeating the last pulses it received. Without a failsafe set on the transmitter the firmware cannot tell that the link is gone. Section 5.2 explains how to make the receiver send an out-of-range throttle value instead, and how to test it. Do not fly before that test passes.

5. Following the project step by step

Keep the propellers off until step 5.6. Each step checks one thing, so that when something is wrong you know exactly where to look.

5.1 Build and flash

Install PlatformIO (the VS Code extension is the easiest way), open the project folder, connect the board over USB and run:

```
pio run -e powerfeather -t upload      # build and flash
pio device monitor                     # console: type h and Enter for help
```

Leave the model still for the first couple of seconds: the gyro is calibrated then. Keep your hands off the roll, pitch and yaw sticks for two more seconds while the firmware learns their resting positions.

5.2 Radio

Type `r`. Roll right, pitch forward and yaw right must all read positive, and the throttle must run from 0.00 to 1.00. Reverse any channel that is wrong. Then set the receiver failsafe:

- *Functions setup → End points*: set the CH3 low end to 120 %.
- *Functions setup → Failsafe → Channel 3*: switch it on, hold the throttle fully down and long-press OK.
- Set the CH3 end point back to 100 %.
- With `r` running, switch the transmitter off. The line must show FAILSAFE within a second.

5.3 Sensor direction and level

Type `i` and tilt the frame by hand: right side down must give positive roll, nose down positive pitch, and turning the nose left a positive yaw rate. If the readings are swapped or reversed, change `IMU_AXIS_MAP`; the setup guide in the repository lists the common mountings. Then put the model on a level surface and type `l` to store that as level.

5.4 Servos

Fit the servo horns so both motors point straight up at centre and fine-tune with `trim l` and `trim r`. Then, still disarmed:

- Tilt the nose down by hand: both motors must tilt back.
- Turn the nose left by hand: the left motor must tilt forward and the right one back.
- Push the pitch stick forward: both motors tilt forward.

If the hand tests fail, swap the servo reverse settings. If only a stick test fails, reverse that stick instead.

5.5 Motors

Calibrate the ESCs once with the `esc_calibration` build, check that the two propellers spin in opposite directions, and use the `o` stream to confirm that tipping the right side down raises the right motor's command.

5.6 First hover

Fit the propellers, choose an open space and raise the throttle slowly. A well-matched setup hovers between 40 % and 60 % throttle. If it needs much more, the controller has no thrust left to correct with, and no amount of tuning will make it steady.

5.7 Tuning

Gains can be changed from the console while disarmed, take effect immediately and are kept after `save`:

```
pid                          show all gains
pid r 0.0016 0.003 0.00004   roll rate P I D
pid p 0.0025 0.004 0         pitch rate P I D
pid y 0.0025 0.003 0         yaw rate P I D
ang r 3.0 / ang p 3.0        self-level strength
ang y 1.5                    heading hold (0 = off)
save                         keep them
```

Tune one axis at a time: roll, then pitch, then yaw. Raise P until the axis starts to wobble and go back to about 70 % of that. Raise I until the model stops leaning or creeping. Use a little D on roll only. To see what the controller is doing, record a log with `tools/flight_log.py`: each line shows the measured and the wanted rate on every axis, the commands and all four outputs.

6. Console reference

Command	Meaning
h	help
s	status line on or off (once a second)
i / r / o / f	stream the IMU, the radio, the outputs or the full flight log (10 per second)
x	stop all output
g	recalibrate the gyro (keep the model still)
l	store the current attitude as level
trim l|r N	servo centre trim in microseconds
pid, pid r|p|y P I D	show or set the rate gains
ang r|p|y P	self-level gain (roll, pitch) or heading hold gain (yaw)
save	write trims, level and gains to flash
defaults	go back to the values in Config.hpp
reboot	restart the board

Every setting that changes the model's behaviour only works while disarmed.

7. Test results

Figure 5 shows 20 seconds of a logged throttle sweep with an earlier version of the controller, and it contains the central result of the project. Roll, driven by the motors, stays within 6 deg of level. Pitch, driven by the servos, swings by up to 25 deg, and its command stays positive for long periods because the centre of gravity sat behind the

rotors and the servos had to hold them tilted forward.

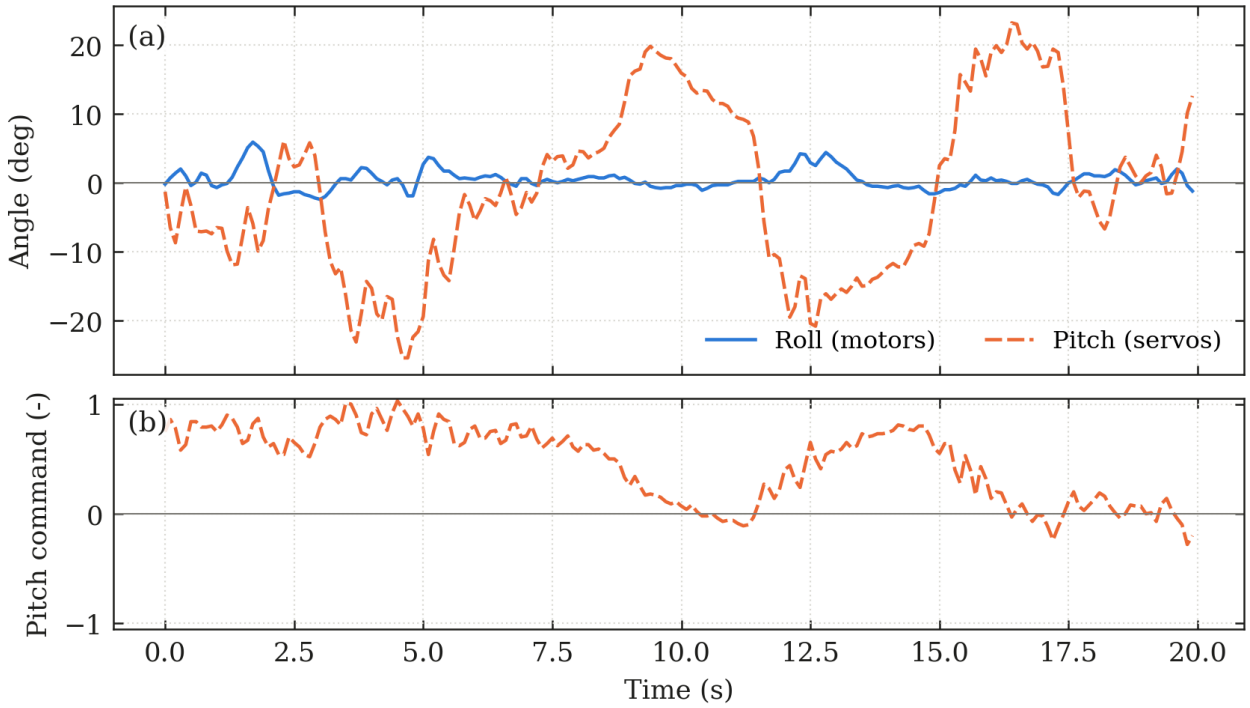


Figure 5. Measured roll and pitch angle and the pitch servo command during a throttle sweep.

Figure 6 compares the complementary filter used earlier with the Mahony filter now in the firmware, on the same simulated hover. The simulation calls the firmware's own estimator. The complementary filter keeps an error of about 1.5 deg caused by the gyro offset; the Mahony filter learns the offset within about 30 s and then stays near zero.

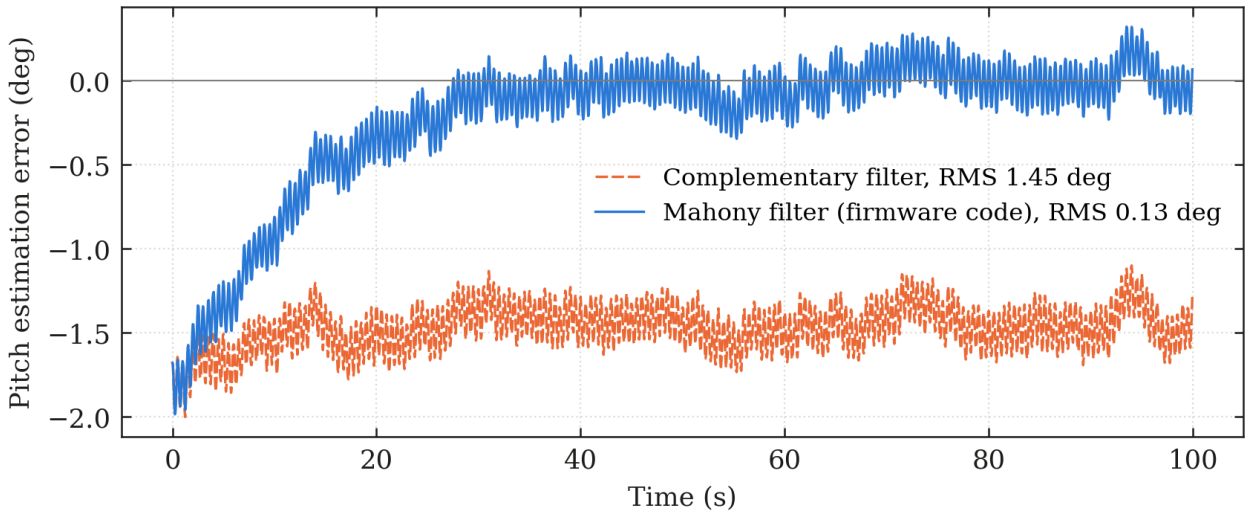


Figure 6. Simulated pitch estimation error, complementary filter and Mahony filter.

Figure 7 shows the effect of the optional phase lead on a simple servo model: 90 % of a 10 deg step is reached after about 30 ms instead of 110 ms. On the real model the lead, together with a higher pitch gain, made the frame vibrate, so it is off by default.

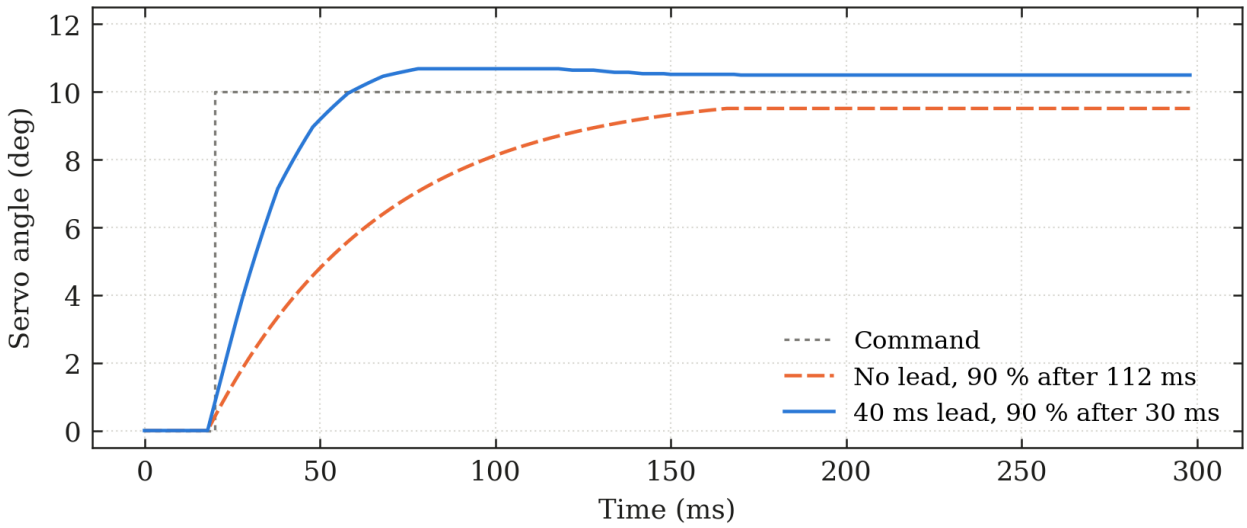


Figure 7. Simulated servo step response with and without the 40 ms phase lead (a model, not a measurement).

Recorded flight of 23 October 2026

Figures 8 to 11 come from a flight recorded with the web page's flight recorder: 22 s at 25 samples per second, of which 20 s airborne at 63 % throttle. The full record is included as docs/data/bicopter_20261023_182347.csv. It was flown with an earlier firmware version in which the hover trim was active, so the pitch target was about 9 deg nose-up rather than level. Table 1 summarises the airborne part.

Quantity (airborne, throttle above 30 %)	Value
Airborne time and throttle	20.2 s at 63 %
Roll angle, mean and standard deviation	+1.1 deg, 5.3 deg
Pitch angle, mean and standard deviation	-18.6 deg, 14.1 deg
RMS rate tracking error: roll / pitch / yaw	16 / 80 / 22 deg/s
Mean ESC pulse, left minus right	+90 us
Motor tilt learned in hover	+9.3 deg (forward)
Samples in which the accelerometer was trusted	44 %

Table 1. Statistics of the recorded flight.

Figure 8 shows the attitude over the whole flight. Roll stays within a few degrees of level apart from short excursions, while pitch repeatedly drops 40 to 55 deg nose-up and recovers, roughly every 2.5 s.

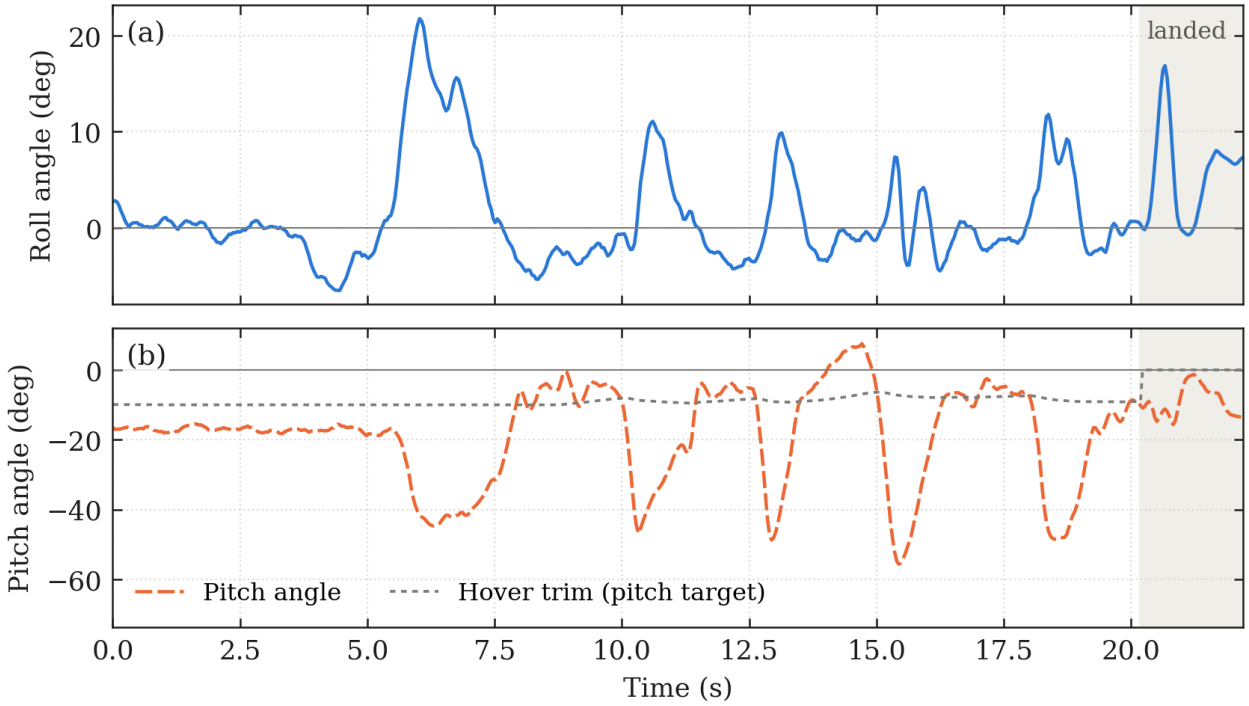


Figure 8. Roll angle (a) and pitch angle with its hover-trim target (b) during the recorded flight.

Figure 9 compares the rate each axis was asked for with the rate the gyro measured, on the same scale. On roll the two curves follow each other closely. On pitch the commanded rate reaches its 90 deg/s limit and stays there for almost two seconds while the model barely responds, and afterwards the measured rate oscillates around the command: the servos cannot produce the pitch moment fast enough.

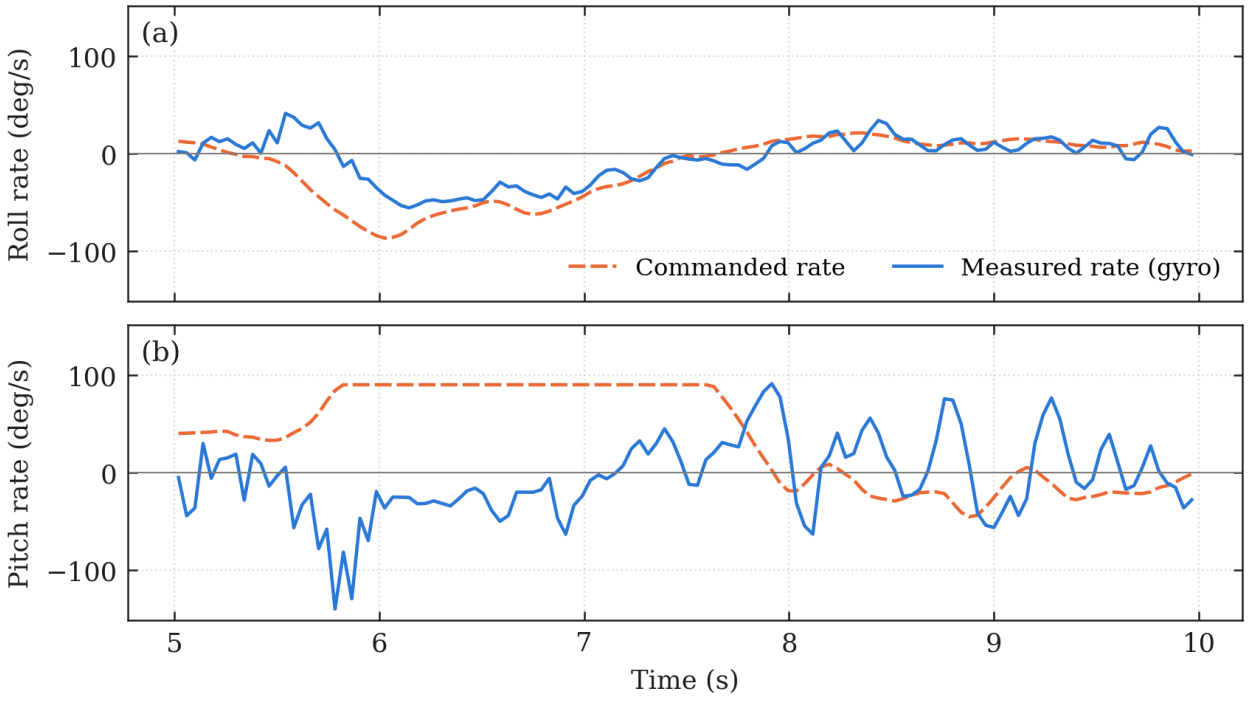


Figure 9. Commanded and measured rate, roll (a) and pitch (b), for 5 s of the flight.

Figure 10 shows the actuator outputs. The left motor runs about 90 us above the right one for most of the flight, so either the centre of gravity was offset towards the left or the left motor and propeller give less thrust; weighing the two sides would tell which. Both servos

stay away from their 1500 us centre, holding the rotors tilted forward: the centre of gravity was also behind the rotor axis, which matches the +9.3 deg hover tilt in Table 1.

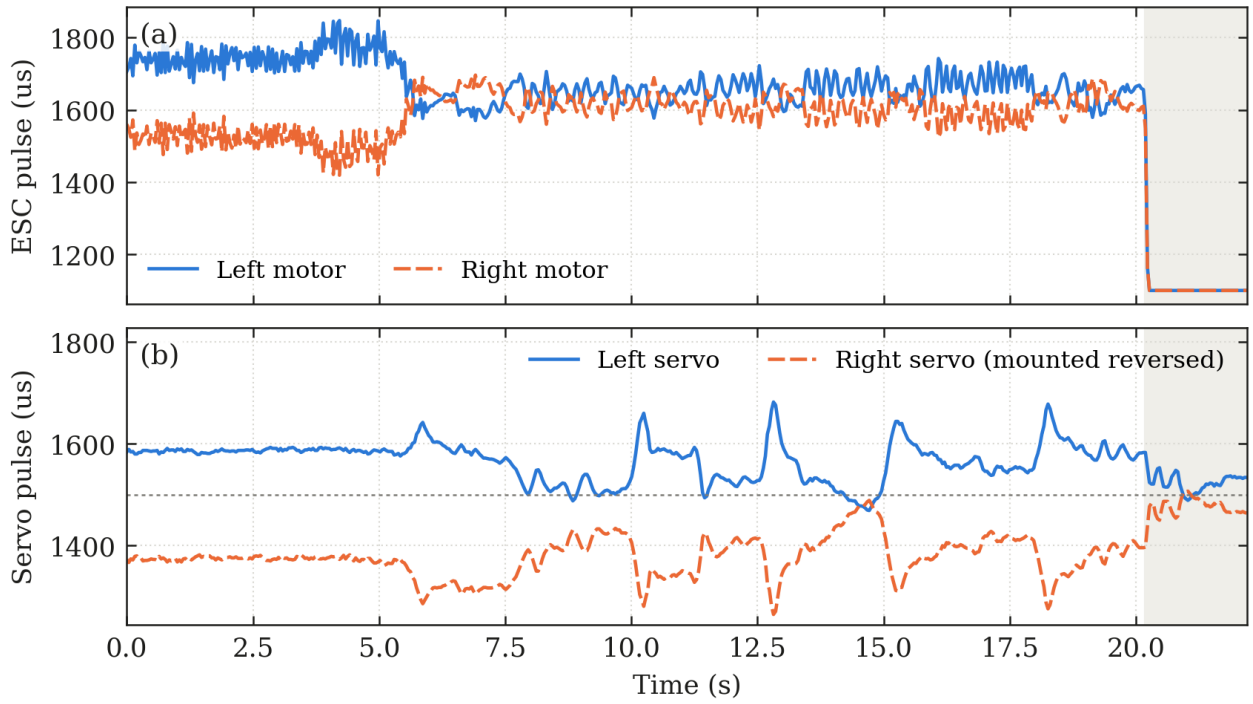


Figure 10. ESC pulses (a) and servo pulses (b) during the recorded flight. The right servo is mounted mirrored, so it moves the opposite way for the same tilt.

Figure 11 condenses the comparison into one number per axis. The pitch axis, driven by the servos, tracks its commanded rate five times worse than the roll axis, driven by the motors.

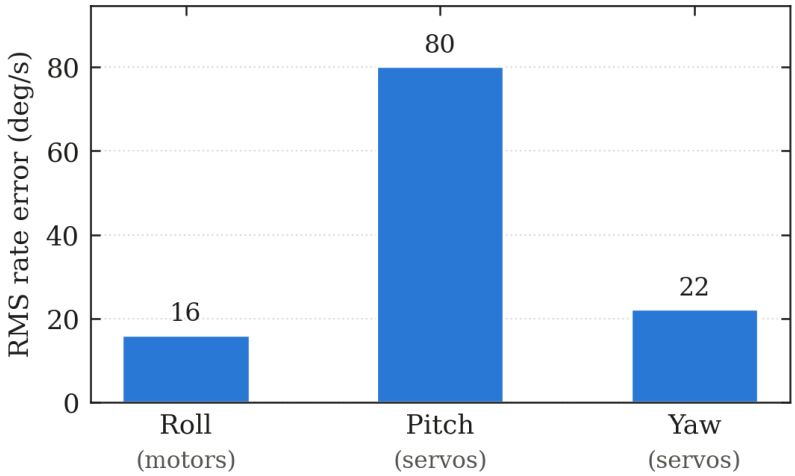


Figure 11. RMS rate tracking error per axis during the airborne part of the recorded flight.

All figures are generated by tools/make_figures.py from the data in docs/data and from the simulations.

Problems found during testing

These came up while the firmware was being brought up on the real model. They are worth reading before your own first test, because most of them look like a tuning problem but are not.

Symptom	Cause	What fixed it
Console printed almost nothing	The PowerFeather's USB buffer holds only 64 bytes, and longer lines were being dropped.	Console output now goes through a 2 KB queue.
Angle never settled, drifted	The accelerometer read 1.42 g at rest, so the filter rejected it.	The Z offset is measured at every start-up and removed.
Roll and pitch swapped	The sensor was mounted rotated 90° from the assumed orientation.	New IMU_AXIS_MAP; a compile-time check stops mirror-image maps.
Model kept turning	The yaw stick rested at 1438 µs because of a transmitter trim.	Stick centres are learned at start-up; yaw has a wider deadband.
Servos slowly moved apart	Raising the throttle nudged the yaw stick, and the yaw I term built up.	Wider yaw deadband and a yaw-I readout in the flight log.
Both servos crept forward	Level was never calibrated (the board read -8° when level).	Level calibration with the l command.
Board stuck in FAULT at boot	The frame was moved during gyro calibration.	Calibration now retries on its own until the model is still.
Upload failed every first time	The USB port reconnects when the chip enters its bootloader.	1200-baud reset in platformio.ini.
Shot forward at take-off	The pitch I term wound up against the ground before lift-off.	Take-off detection: I terms only switch on at the lift-off throttle.
Forward/back swing, needs a lot of room	Slow servos on the pitch axis, and extra controller layers that made fast servo starts worse.	Simple pitch PID, CG low under the rotors, front-to-back balance.

8. Limitations

- **No position hold.** The model holds angle and heading. Wind or a small level error makes it slide slowly, and the pilot has to correct.
- **Servo quality.** The MG996R is slow and has free play. It sets the limit on how precisely pitch and yaw can be held. A digital metal-gear servo running at 200 Hz would be a clear improvement.
- **Thrust margin.** During testing the model needed close to full throttle. Below about 60 % hover throttle the controller has the headroom it needs; above that, every correction fights saturation.
- **Yaw reference.** Without a magnetometer the heading slowly wanders with gyro drift. The firmware re-zeroes the gyro whenever the model sits still, which keeps this small.

9. Where it could go next

- An optical-flow sensor (for example the PMW3901) and a laser rangefinder (for example the VL53L1X) for position and height hold indoors.
- Battery voltage measurement through a divider on a spare ADC pin, to keep the throttle response the same as the battery runs down and to warn before it is empty.
- Logging to the PowerFeather's flash for later analysis, instead of over USB.
- Feed-forward from the roll motor command into yaw, to cancel the torque coupling before the gyro sees it.

10. Resources and references

The firmware draws on the datasheets and documentation for the parts used, on standard references for attitude estimation and feedback control, and on several open source flight controllers whose approaches were studied while designing this one.

[1] Espressif Systems, *ESP32-S3 Technical Reference Manual* (chapters on MCPWM and LED PWM controller). https://www.espressif.com/en/support/documents/technical-documents

[2] Espressif Systems, *Arduino-ESP32 documentation*. https://docs.espressif.com/projects/arduino-esp32

[3] PowerFeather, *ESP32-S3 PowerFeather documentation and pinout*. https://powerfeather.dev

[4] InvenSense (TDK), *MPU-6000 and MPU-6050 Product Specification* (PS-MPU-6000A-00) and *Register Map and Descriptions* (RM-MPU-6000A-00).

[5] PlatformIO, *PlatformIO documentation*. https://docs.platformio.org

[6] FlySky, *FS-i6 Digital Proportional Radio Control System, user manual*.

[7] R. Mahony, T. Hamel and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group", *IEEE Transactions on Automatic Control*, vol. 53, no. 5, 2008.

[8] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays", technical report, University of Bristol, 2010.

[9] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008.

[10] Q. Quan, *Introduction to Multicopter Design and Control*, Springer, 2017.

[11] Y. Qin, W. Xu, A. Lee and F. Zhang, "Gemini: A compact yet efficient bi-copter UAV for indoor applications", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, 2020.

[12] ArduPilot, *Tilt rotor planes* (bicopter frame class). https://ardupilot.org/plane/docs/guide-tilt-rotor.html

[13] P. Cook, *PlainFlightController*. https://github.com/plainFlight/plainFlightController

[14] N. Rehm, *dRehmFlight VTOL flight controller*. https://github.com/nickrehm/dRehmFlight

[15] R. Lopez, *ESP-FC, a flight controller for ESP32*. https://github.com/rtlopez/esp-fc

[16] Espressif Systems, *ESP-Drone*. https://github.com/espressif/esp-drone

[17] Betaflight, *Betaflight documentation* (PID tuning and filtering). https://betaflight.com/docs